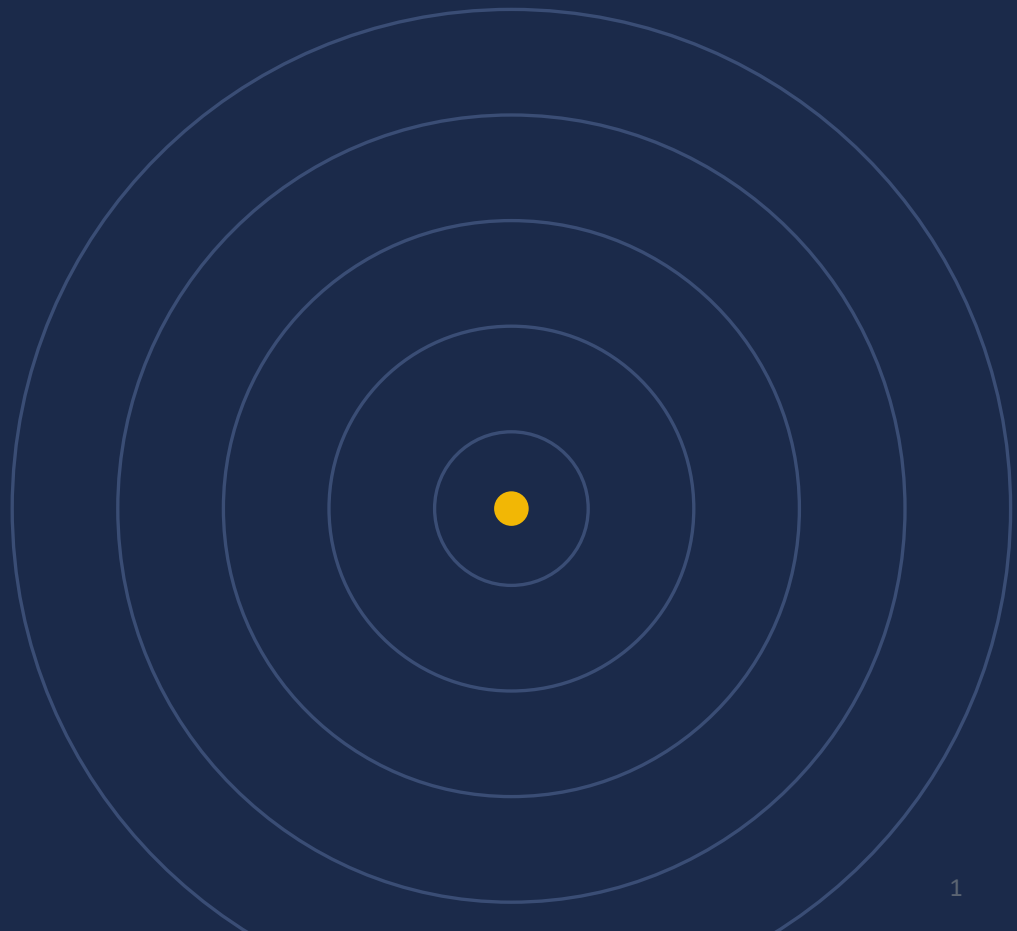
Gradient Descent

How the loss surface guides every step of learning

Optimizing weight (w) and bias (b) by following the gradient of L(w, b)



Click through to watch the optimizer descend, step by step

The Idea Behind Gradient Descent

A simple model predicts with a weight w and a bias b. We measure how wrong it is with a loss function, then adjust w and b to make the loss smaller.

The moving parts

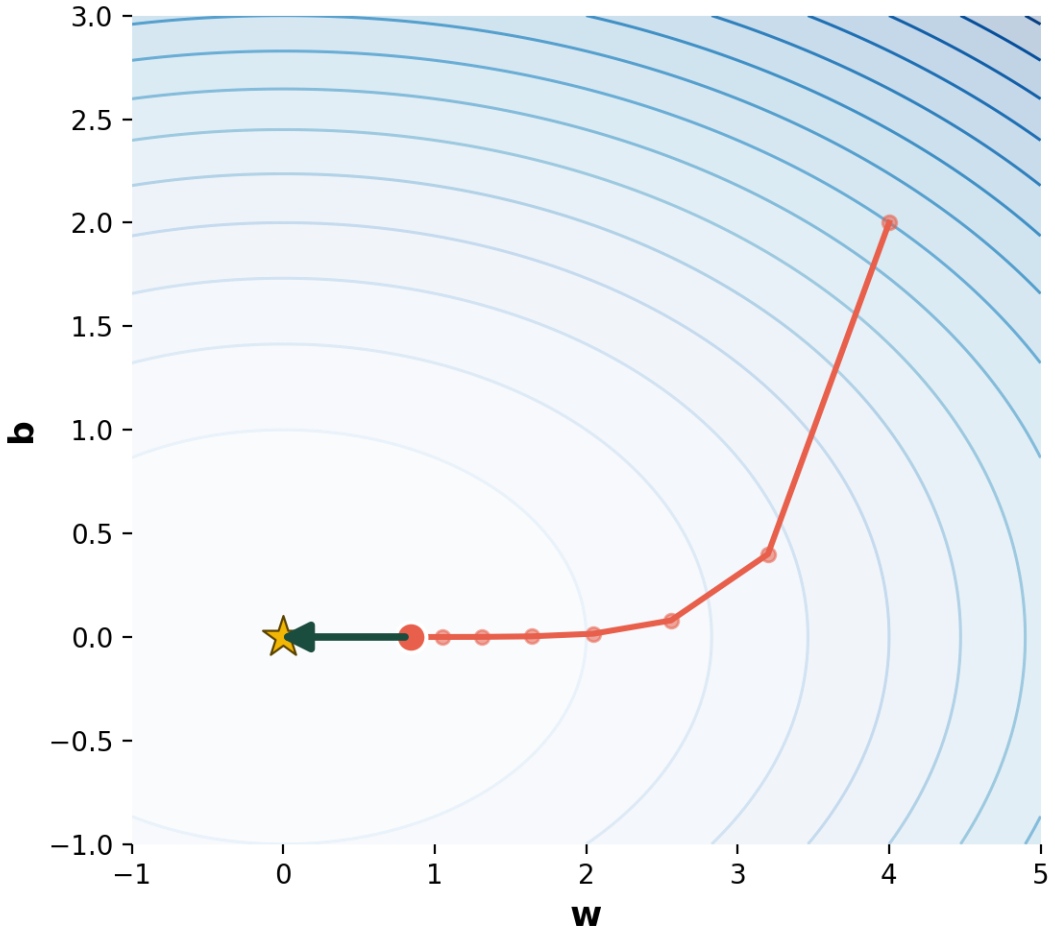
Loss function	$L(w, b) = w^2 + 4b^2$	*A convex bowl — one global minimum at (0, 0)*
Gradient	$\nabla L = \left(\partial L/\partial w , \partial L/\partial b \right) = (2w, 8b)$	*Points in the direction of steepest increase*
Update rule	$w \leftarrow w - \alpha \cdot \partial L/\partial w$ $b \leftarrow b - \alpha \cdot \partial L/\partial b$	*Step opposite the gradient, scaled by learning rate α*
Learning rate	$\alpha = 0.1$	*Controls how big each step is*

What to watch for

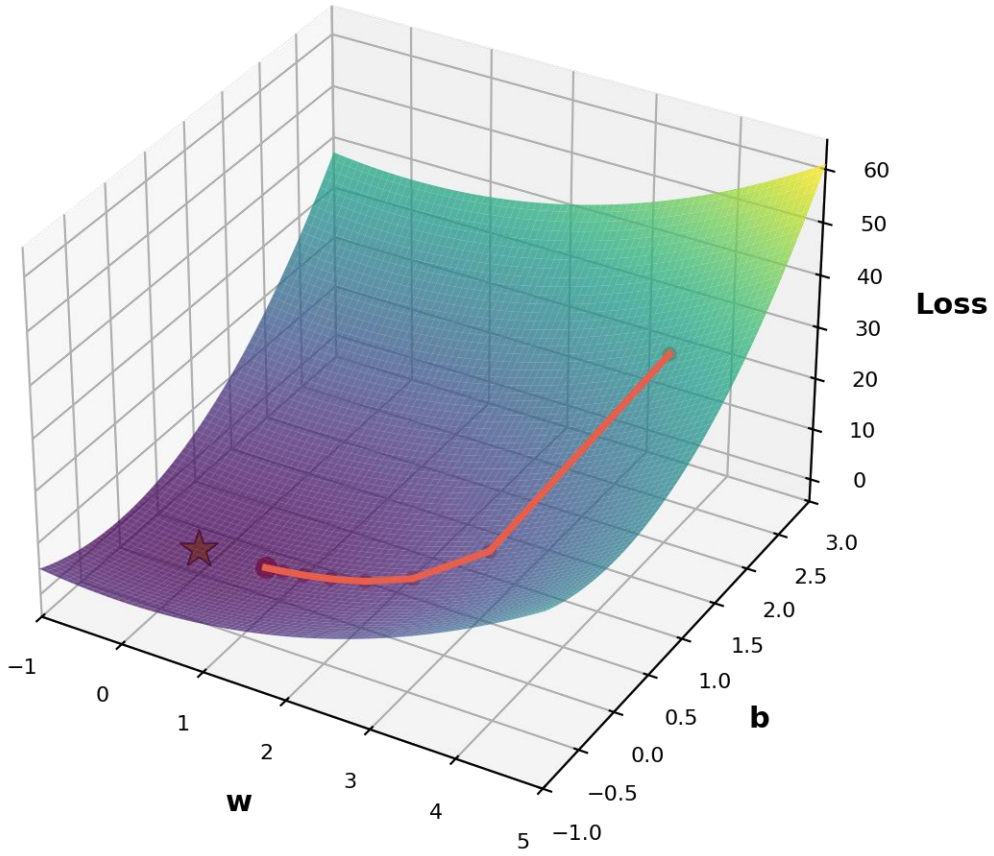
- The coral dot is the current (w, b) — it starts far from the minimum.
- The dark green arrow is the negative gradient — the direction of steepest descent.
- b has a steeper curvature (coefficient 4) than w, so it races to 0 first.
- w lags behind — this is why a single fixed learning rate is a compromise, motivating momentum & adaptive methods.
- The gold star marks the minimum, where the loss is lowest.

Step 7

Click "next slide" to take the next gradient descent step



Contour view



3D surface view

w

0.8389

b

0

Loss L(w,b)

0.7037

∂L/∂w

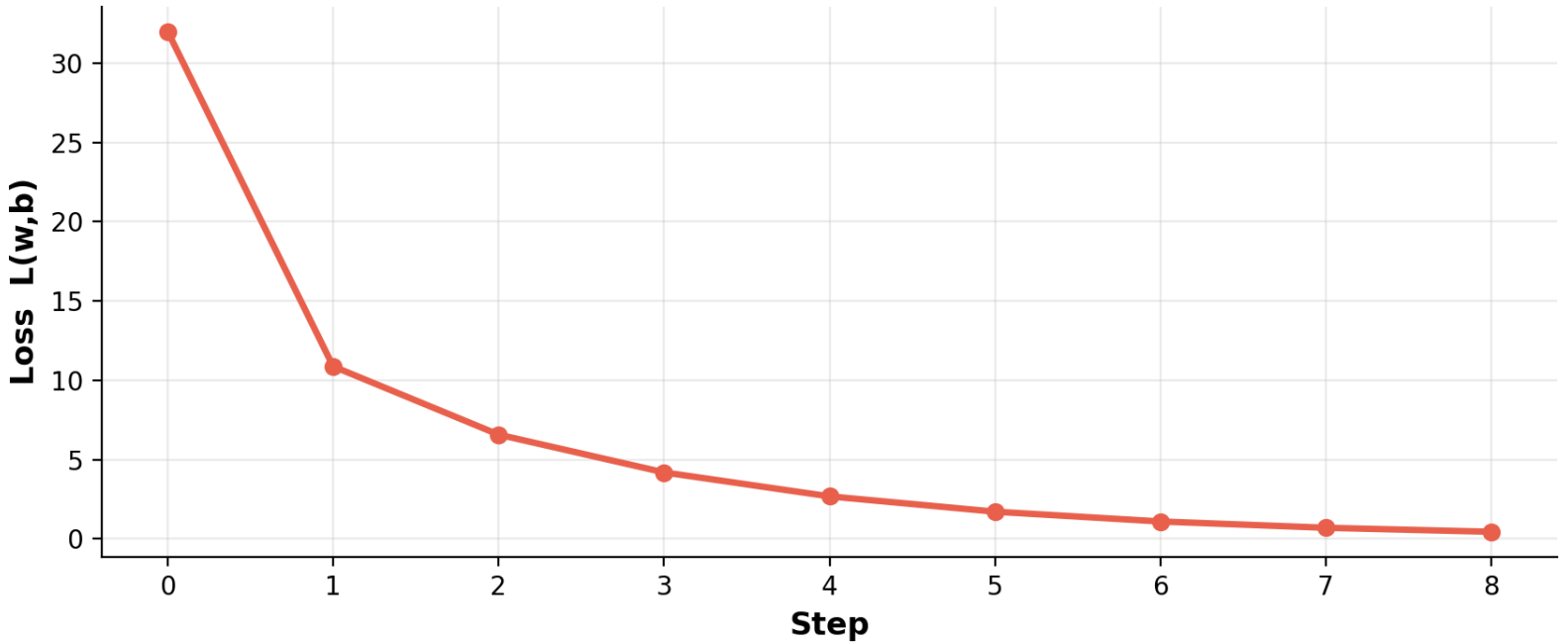
1.6777

∂L/∂b

0.0002

Loss Decreases With Every Step

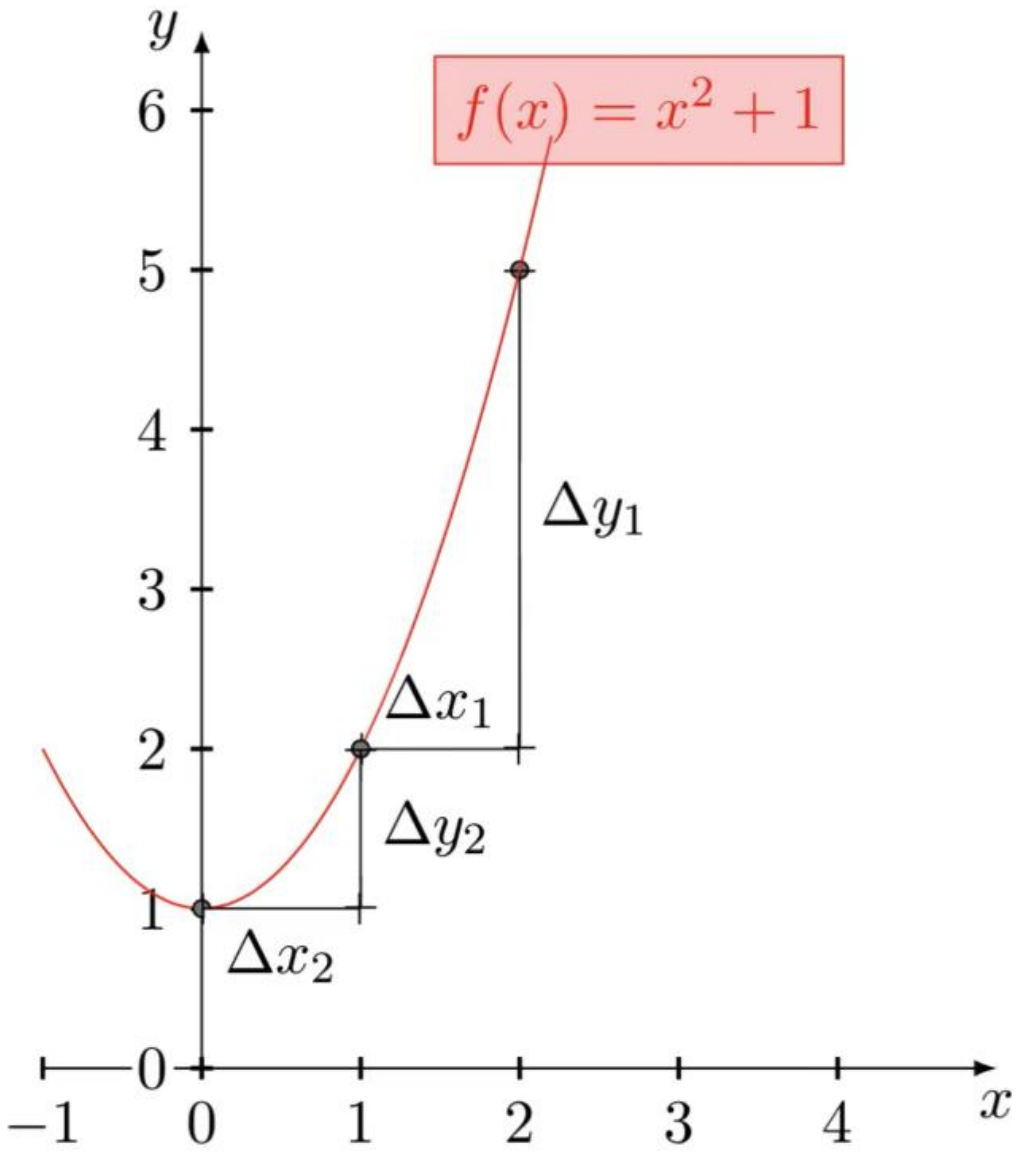
Each gradient step moves (w, b) to lower ground on the loss surface — the loss curve is monotonically decreasing.



Update rule used throughout: $w \leftarrow w - 0.1 \cdot (2w)$ $\quad$ $b \leftarrow b - 0.1 \cdot (8b)$

Key takeaways

- Gradient descent always moves opposite the gradient — downhill on the loss surface.
- Step size = learning rate × gradient magnitude.
- Directions with higher curvature (b here) converge faster than shallow ones (w).
- This imbalance is exactly what momentum, RMSprop, and Adam are designed to fix.



$$w \leftarrow w - \alpha \cdot \partial L / \partial w$$

$$b \leftarrow b - \alpha \cdot \partial L / \partial b$$

How we compute gradients ?

or

What data we use to compute gradients ?

Batch Gradient Descent

Every update so far in this deck has used batch gradient descent: the full training set is used to compute one true gradient before (w, b) ever moves.

Per-step math

Gradient	$\nabla L = (1/N)\cdot\Sigma\ \nabla L_i(w,b)$	*Averaged over all N training examples*
Update	$w \leftarrow w - \alpha\cdot\partial L/\partial w$	$b \leftarrow b - \alpha\cdot\partial L/\partial b$
Batch size	`m = N` (the entire dataset)	*One gradient computed per full pass over the data (an epoch)*
Convergence	Same path shown in slides 3–11	*Smooth, monotonic descent — this is exactly the update rule used earlier*

Trade-offs

- Uses the true gradient at every step, so the path down is smooth and stable, with no zig-zag from sampling noise.
- For a convex loss like $L(w,b) = w^2 + 4b^2$, batch GD is guaranteed to converge to the global minimum.
- Only one update per epoch — on large datasets, each step is slow because it waits on the whole pass through the data.
- Needs the full dataset available at once, which becomes impractical as N grows — motivating SGD and mini-batches.

Stochastic Gradient Descent (SGD)

Instead of waiting on the whole dataset, SGD updates (w, b) after every single training example — a noisier but much faster step.

Per-step math

Gradient	$\nabla L \approx \nabla L_i(w,b)$	*Estimated from one randomly chosen example i*
Update	$w \leftarrow w - \alpha \cdot \partial L_i / \partial w$	$b \leftarrow b - \alpha \cdot \partial L_i / \partial b$
Batch size	m = 1 (a single example)	*N updates per epoch instead of one — one per training example*
Path	Zig-zags toward the minimum	*Unlike the smooth curves in slides 3–11, each step can move slightly off-course*

Trade-offs

- Each step is nearly free to compute, so SGD can take far more steps than batch GD in the same amount of time.
- The noisy path can actually help — it lets the optimizer escape shallow local minima or saddle points that trap batch GD.
- The loss no longer decreases every single step — it wobbles even near the minimum, since each example is only a rough estimate.
- A decaying learning rate is usually needed, since a fixed α leaves the optimizer bouncing around the minimum forever.

Mini-Batch Gradient Descent

A middle ground between the two: average the gradient over a small random batch of examples, not the whole dataset and not just one.

Per-step math

Gradient	$\nabla L \approx (1/m)\cdot\Sigma\ \nabla L_i(w,b)$	*Averaged over a small random batch of m examples*
Update	$w \leftarrow w - \alpha\cdot\partial L/\partial w$	$b \leftarrow b - \alpha\cdot\partial L/\partial b$
Batch size	$m \approx 32–256$ (typical range, tunable)	*A hyperparameter that trades off noise against speed and memory*
Hardware fit	Batches vectorize well	*Fixed-size batches map cleanly onto GPU/vectorized computation*

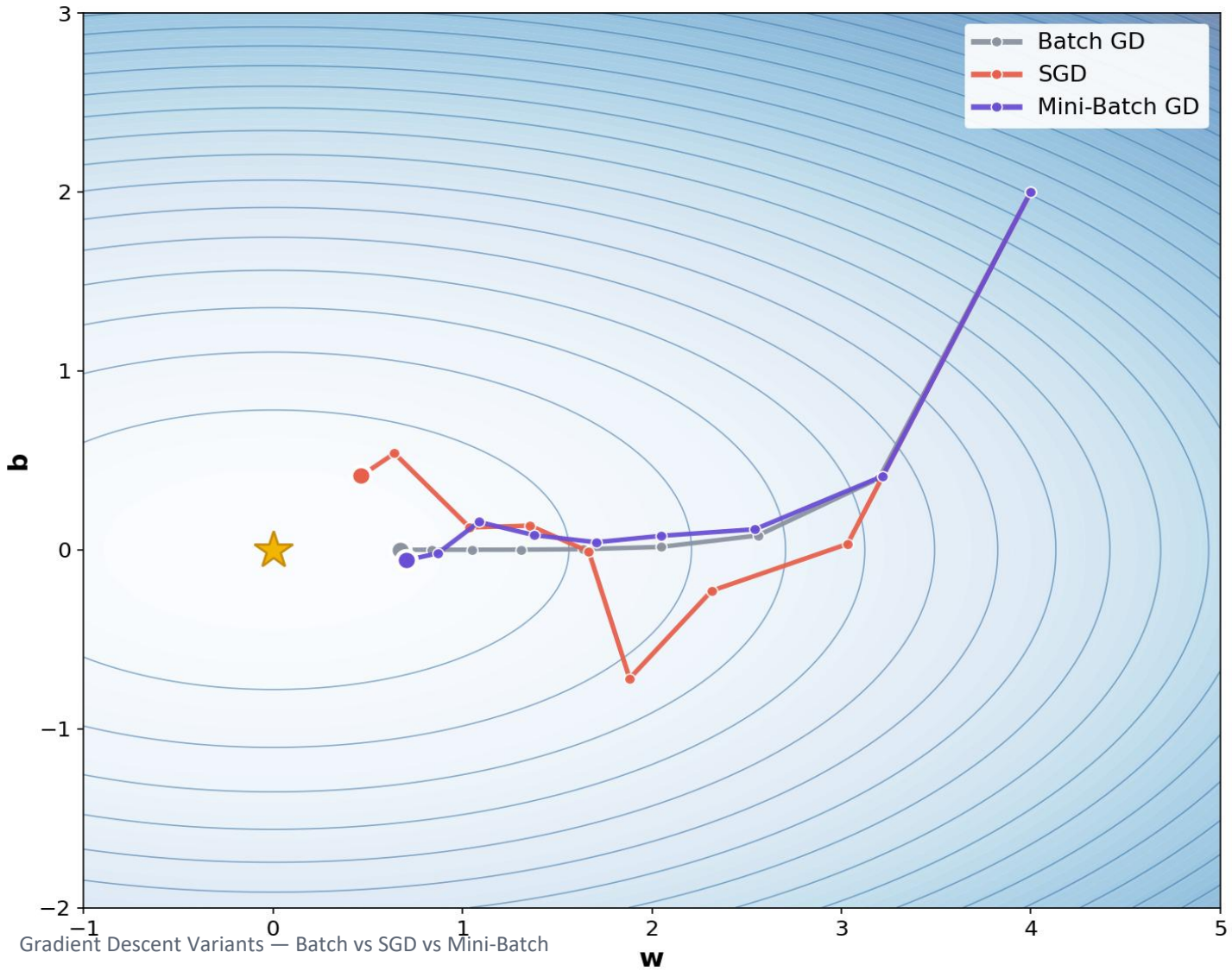
Trade-offs

- Smooths out most of the noise seen in SGD, while staying far cheaper per step than a full batch GD pass.
- Larger batches converge more smoothly but need more memory; smaller batches add helpful noise but more variance.
- This is the default in practice — nearly every deep learning optimizer (SGD with momentum, RMSprop, Adam) runs on mini-batches underneath.
- Choosing m is itself a tuning decision — too small drifts back toward SGD's noise, too large drifts back toward batch GD's cost.

Step 8

Batch GD vs SGD vs Mini-Batch GD — same learning rate (α = 0.1), click to advance

9 / 9

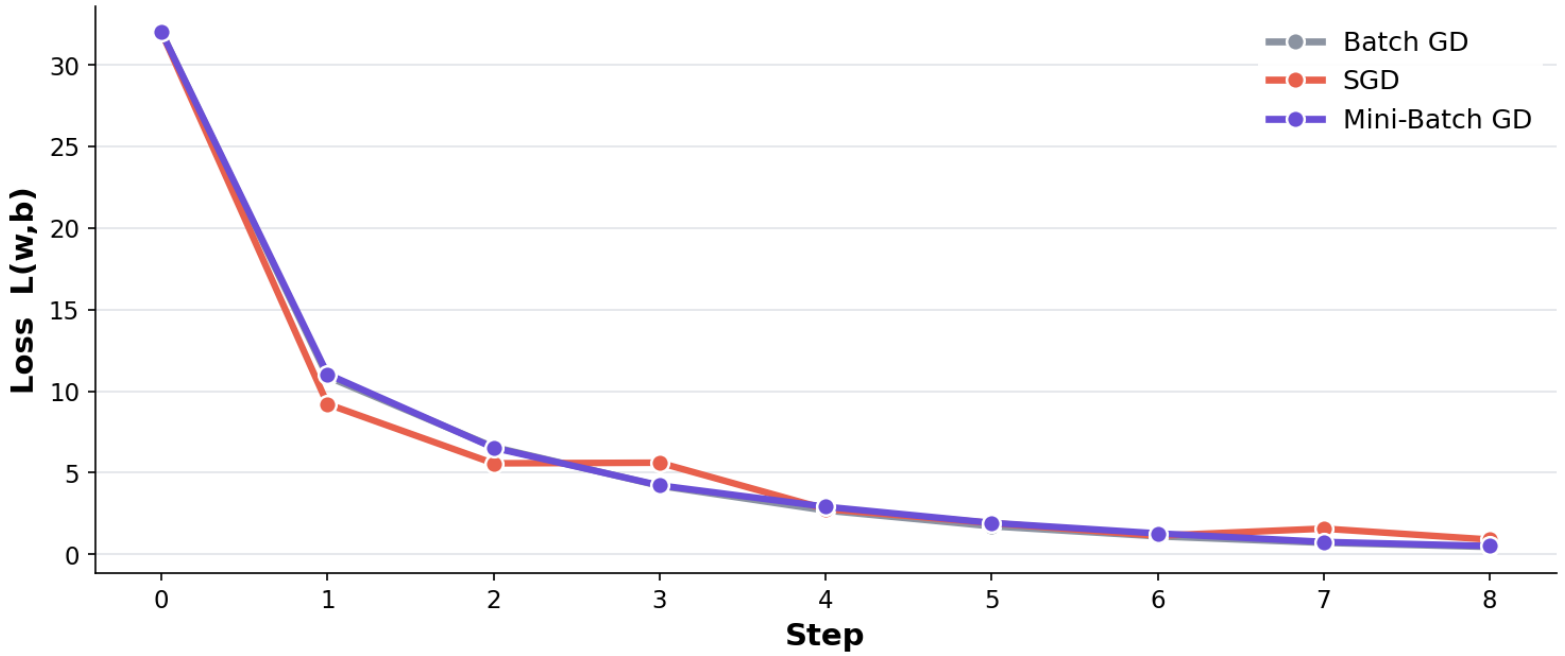


	BATCH GD	SGD	MINI-BATCH GD
w	0.6711	0.4652	0.7023
b	0	0.4136	-0.0587
Loss L(w,b)	0.4504	0.9006	0.5071
Batch size (m)	N	1	20

Same starting point (w=4, b=2) and learning rate α=0.1 for all three.

Batch vs SGD vs Mini-Batch — Summary

Same learning rate, same starting point, same 8 steps — batch GD lands at a loss of 0.4504; mini-batch lands close behind at 0.5071; SGD, taking one noisy example at a time, is still at 0.9006.



$$\text{Batch: } \nabla L=(1/N)\Sigma\nabla L_i \qquad \text{SGD: } \nabla L\approx\nabla L_i \qquad \text{Mini-batch: } \nabla L\approx(1/m)\Sigma\nabla L_i \qquad \alpha=0.1$$

Key takeaways

- Batch GD moves smoothly and predictably — it always uses the true gradient over all N examples.
- SGD is noisiest: one example per step is cheap, but the path zig-zags and can even overshoot the minimum.
- Mini-batch GD (m=20 here) tracks the batch path closely while costing far less than a full pass per step.
- This is exactly why mini-batches are the default in practice — most of batch GD's stability at a fraction of the cost.

Batch vs SGD vs Mini-Batch — Epochs, steps

- 1 epoch = one pass over the entire data
- 1 step = one update of the parameters
- N = number of data points
- B = mini-batch size

Algorithm	# of steps in one epoch
Batch gradient descent	1
Stochastic gradient descent	N
Mini-Batch gradient descent	N/B

Momentum-Based Gradient Descent

Vanilla gradient descent only looks at the current slope. Momentum also remembers which way it has been moving — like a ball rolling downhill that picks up speed.

Update rule

Velocity (w)	`v_w ← β·v_w + α·∂L/∂w`	*Exponential moving average of past gradients*
Velocity (b)	`v_b ← β·v_b + α·∂L/∂b`	*Same idea, applied to b*
Update	`w ← w − v_w` `b ← b − v_b`	*Move by the velocity, not just the raw gradient*
Momentum coeff.	`β = 0.7` (typical range 0.8–0.99)	*How much of the past velocity carries forward*

Why it helps

- In directions where the gradient keeps pointing the same way (w, here), velocity builds up and the step size grows — much faster progress.
- In directions where the gradient flips sign (b, here, near the steep walls), velocity partially cancels itself out, damping the zig-zag over time.
- Net effect: fewer wasted steps, faster travel through shallow, consistent-slope regions — exactly where vanilla GD crawls.
- Trade-off: momentum can overshoot the minimum before settling, visible as a slight dip past zero in the next slides.

How we use gradients ?

or

Can we come up with a better update rule?

Momentum-based Gradient Descent

Momentum based Gradient Descent Update Rule

$$v_t = \gamma * v_{t-1} + \eta \nabla w_t$$

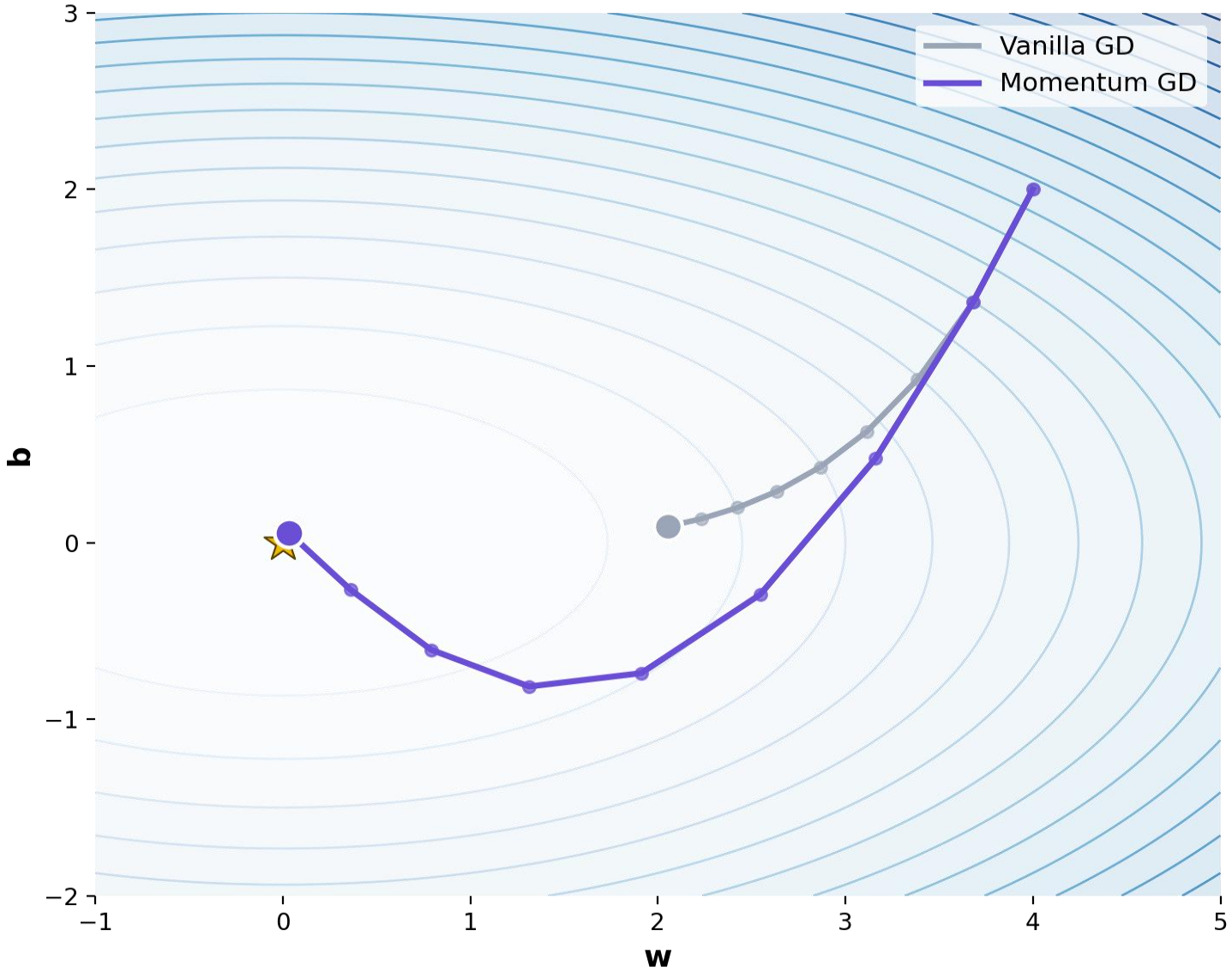
$$w_{t+1} = w_t - v_t$$

$$w_{t+1} = w_t - \gamma * v_{t-1} - \eta \nabla w_t$$

Step 8

Vanilla GD vs Momentum GD — same learning rate (α = 0.04), click to advance

9 / 9



VANILLA GD	MOMENTUM GD
w	
2.0529	0.0317
b	
0.0914	0.0564
Loss L(w,b)	
4.2477	0.0137

Momentum is ahead by

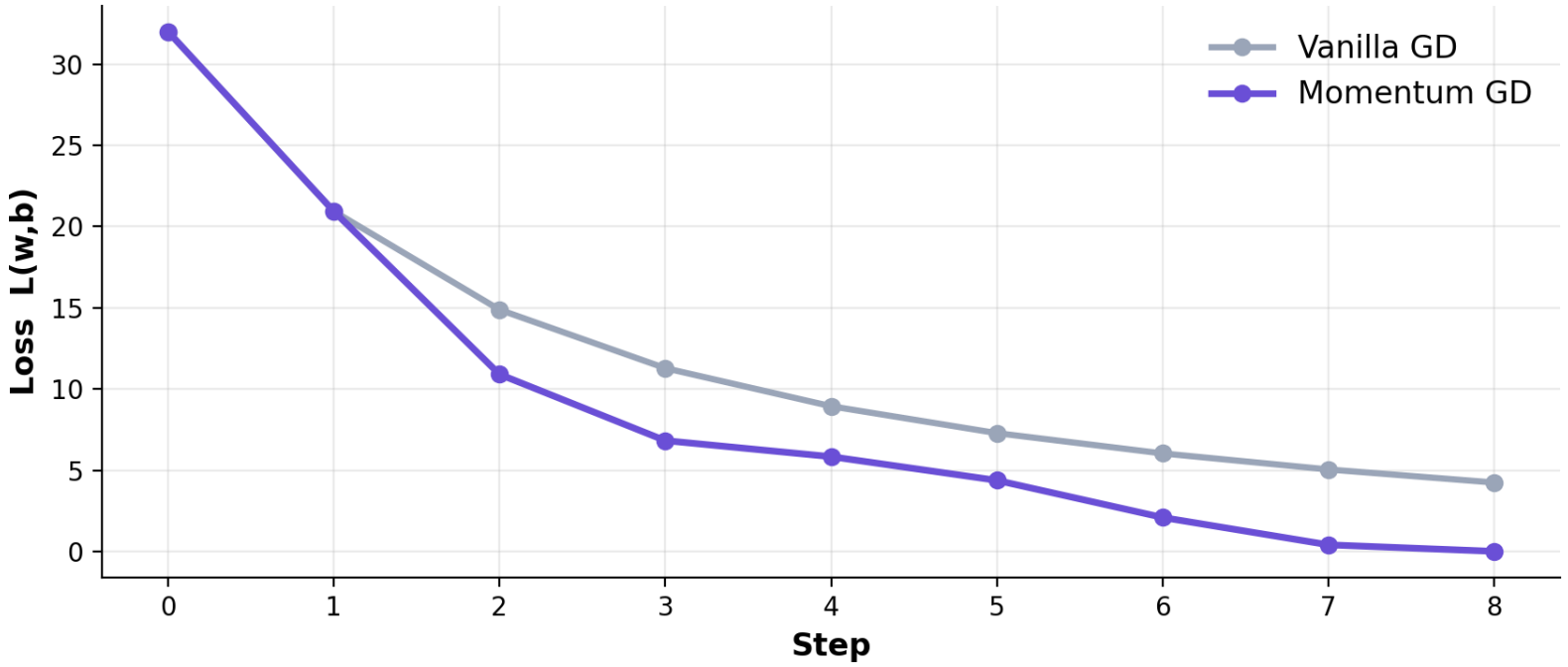
310.1× lower loss

Velocity (w, b)

`(0.3295, -0.3232)`

Momentum Wins the Race

Same learning rate, same starting point, same 8 steps — momentum reaches a loss of 0.01 while vanilla GD is still at 4.25.



$$v \leftarrow \beta \cdot v + \alpha \cdot \nabla L(w,b) \qquad (w,b) \leftarrow (w,b) - v \qquad \alpha=0.04,\ \beta=0.7$$

Key takeaways

- Momentum accelerates travel through shallow, consistent-gradient regions — the exact case where plain GD is slow.
- It dampens oscillation in steep directions over time, by averaging out gradients that flip sign.
- It can overshoot the minimum before settling — visible as the dip past zero in b.
- Same idea underlies Nesterov Momentum, RMSprop, and Adam — each refines how the update remembers the past.

Momentum-based gradient descent

- Momentum based gradient descent oscillates in and out of the minima valley (u-turns)
- Despite these u-turns it still converges faster than vanilla gradient descent

$$v_t = \gamma \cdot v_{t-1} + \eta \nabla w_t$$

$$w_{t+1} = w_t - v_t$$

Momentum based Gradient Descent Update Rule

$$v_t = \gamma * v_{t-1} + \eta \nabla w_t$$

$$w_{t+1} = w_t - v_t$$

$$v_0 = 0$$

$$v_1 = \gamma \cdot v_0 + \eta \nabla w_1 = \eta \nabla w_1$$

$$v_2 = \gamma \cdot v_1 + \eta \nabla w_2 = \gamma \cdot \eta \nabla w_1 + \eta \nabla w_2$$

$$v_3 = \gamma \cdot v_2 + \eta \nabla w_3 = \gamma(\gamma \cdot \eta \nabla w_1 + \eta \nabla w_2) + \eta \nabla w_3$$

$$= \gamma \cdot v_2 + \eta \nabla w_3 = \gamma^2 \cdot \eta \nabla w_1 + \gamma \cdot \eta \nabla w_2 + \eta \nabla w_3$$

$$v_4 = \gamma \cdot v_3 + \eta \nabla w_4 = \gamma^3 \cdot \eta \nabla w_1 + \gamma^2 \cdot \eta \nabla w_2 + \gamma \cdot \eta \nabla w_3 + \eta \nabla w_4$$

$$\vdots$$

$$v_t = \gamma \cdot v_{t-1} + \eta \nabla w_t = \gamma^{t-1} \cdot \eta \nabla w_1 + \gamma^{t-2} \cdot \eta \nabla w_1 + \ldots + \eta \nabla w_t$$

Nesterov Accelerated gradient descent

Momentum based Gradient Descent Update Rule

$$v_t = \gamma * v_{t-1} + \eta \nabla w_t$$

$$w_{t+1} = w_t - v_t$$

$$w_{t+1} = w_t - \gamma * v_{t-1} - \eta \nabla w_t$$

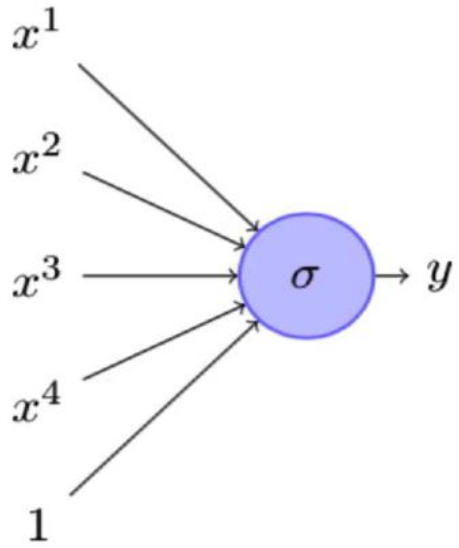
NAG Update Rule

$$w_{temp} = w_t - \gamma * v_{t-1}$$

$$w_{t+1} = w_{temp} - \eta \nabla w_{temp}$$

$$v_t = \gamma * v_{t-1} + \eta \nabla w_{temp}$$

Why Adaptive Learning Rate ?



$$w \leftarrow w - \alpha \cdot \partial L/\partial w$$

$$b \leftarrow b - \alpha \cdot \partial L/\partial b$$

$$\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$$
$$\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$$

$$y = f(x) = \frac{1}{1+e^{-(\mathbf{w}\cdot\mathbf{x}+b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

Can we have a different learning rate for each parameter which takes care of the frequency of features ?

Sparse features – Large learning rate
Dense features – Small learning rate

Adagrad

Intuition: Decay the learning rate for parameters in proportion to their update history (fewer updates, lesser decay)

Adagrad

$$v_t = v_{t-1} + (\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{(v_t)} + \epsilon} \nabla w_t$$

Advantage

- Parameters corresponding to sparse features get better updates

Disadvantage

- The learning rate decays very aggressively as the denominator grows (not good for parameters corresponding to dense features)

RMSProp

Intuition: Why not decay the denominator and prevent its rapid growth ?

RMSProp

$$v_t = \beta * v_{t-1} + (1 - \beta)(\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{(v_t)} + \epsilon} \nabla w_t$$

Adam Optimizer

Momentum based Gradient Descent Update Rule

$$v_t = \gamma * v_{t-1} + \eta \nabla w_t$$

$$w_{t+1} = w_t - v_t$$

RMSProp

$$v_t = \beta * v_{t-1} + (1 - \beta)(\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{(v_t)} + \epsilon} \nabla w_t$$

Adam

$$m_t = \beta_1 * v_{t-1} + (1 - \beta_1)(\nabla w_t)$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2)(\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{(v_t)} + \epsilon} m_t$$

$$m_t = \frac{m_t}{1 - \beta_1^t}$$

$$v_t = \frac{v_t}{1 - \beta_2^t}$$

Bias Correction
At beginning of training ensures updates do not behave in weird manner